

Sprint 3 Structured Notes: Bash File and Folder Commands

1. Current Progress

You are now in **Sprint 3** of the Relational Databases learning path.

The full learning path is:

Tools → Terminal → Bash → Bash Scripting → Git → PostgreSQL → SQL → Normalization, Security, Performance

So far, you have completed the foundation stages:

Sprint 1: Tools and VS Code

In Sprint 1, you learned how to prepare your coding environment.

You learned about:

- VS Code
- Code editors and IDEs
- Workspace
- Project folders
- `index.html`
- Live Server
- Basic VS Code shortcuts
- Beginner extensions

Simple meaning:

Sprint 1 helped you prepare the place where you will write and manage your project files.

Sprint 2: Terminal, Shell, and Command Line Basics

In Sprint 2, you learned what happens when you type commands.

You learned about:

- Terminal
- Command line
- Shell
- Bash
- Command history

- Auto-complete
- Clearing the terminal screen
- Stopping a running command

Simple meaning:

Sprint 2 helped you understand the environment where commands are typed and run.

Sprint 3: Bash File and Folder Commands

Now you are learning how to use Bash commands to work with files and folders.

Simple meaning:

Sprint 3 teaches you how to move around folders, create files, read files, rename files, copy files, and delete files using commands.

2. Main Goal of Sprint 3

The main goal of Sprint 3 is:

Navigate and manage files using Bash commands.

By the end of Sprint 3, you should be able to:

- Check your current folder
- Move between folders
- Understand paths
- Create folders
- Create files
- Write text into files
- Add more text to existing files
- Read file content
- Rename files
- Move files
- Copy files
- Delete files safely
- Use command flags and options
- Ask for command help

Simple meaning:

You should be able to control files and folders from the terminal without depending only on mouse clicks.

3. Why Sprint 3 Is Important

Sprint 3 is very important because file and folder commands are used everywhere in development.

Later, you will use these skills when working with:

- Bash scripts
- Git repositories
- GitHub projects
- SQL files
- PostgreSQL project folders
- README files
- Backend project folders
- Database scripts
- Real software projects

Example:

Before using Git, you need to know how to create a project folder.

Before writing SQL scripts, you need to know how to create `.sql` files.

Before running Bash scripts, you need to know how to move to the correct folder.

Simple meaning:

If you understand Sprint 3 well, Git, Bash scripting, SQL files, and PostgreSQL projects will become easier later.

4. What Is Bash?

Bash means:

Bourne Again Shell

Bash is one type of shell.

From Sprint 2, remember:

Term	Simple Meaning
Terminal	The app or window you open
Command line	The place where you type commands

Term	Simple Meaning
Shell	The program that understands and runs commands
Bash	One common type of shell

Simple meaning:

Bash is a shell that understands commands such as `pwd`, `cd`, `ls`, `mkdir`, `touch`, `cp`, `mv`, **and** `rm`.

You can use Bash in:

- Linux terminal
- macOS terminal
- Git Bash on Windows
- VS Code terminal when configured with Bash
- Many servers

5. What Is a Working Directory?

A **working directory** is the folder where your terminal is currently located.

Think of your terminal like a person standing inside a folder.

The folder where the terminal is standing right now is called the working directory.

Example:

```
/home/user/my-project
```

If your terminal is currently inside this folder, then this is your working directory.

Simple meaning:

Working directory = the folder where your terminal is currently standing.

6. `pwd` Command

The command `pwd` means:

print working directory

It shows the full path of the folder where you are now.

Command:

```
pwd
```

Example output:

```
/home/maleesha/my-project
```

This tells you that your terminal is currently inside the `my-project` folder.

Simple meaning:

`pwd` **shows where you are right now.**

When should you use `pwd` ?

Use `pwd` when:

- You are not sure where you are
- You want to check your current folder before creating files
- You want to check your current folder before deleting files
- You are following instructions and want to avoid mistakes

Beginner habit:

Before creating, moving, copying, or deleting files, run:

```
pwd
```

This helps you avoid working in the wrong folder.

7. What Is a Path?

A **path** tells the computer where a file or folder is located.

Simple meaning:

A path is the address of a file or folder.

Example path:

```
/home/maleesha/projects/my-project
```

This path tells the computer where the `my-project` folder is located.

There are three important path ideas in Sprint 3:

1. Absolute path
 2. Relative path
 3. Parent directory using `..`
-

8. Absolute Path

An **absolute path** is the full address of a file or folder.

It starts from the root of the system.

Example:

```
/home/maleesha/projects/my-project
```

This is like giving the full address of a house.

Simple meaning:

Absolute path = full location from the beginning of the system.

Example use

```
cd /home/maleesha/projects/my-project
```

This tells the terminal exactly where to go.

It does not depend on your current folder.

9. Relative Path

A **relative path** is a location based on where you are now.

Example:

Imagine your terminal is already inside:

```
/home/maleesha
```

Inside this folder, there is another folder called:

```
projects
```

You can go into it using:

```
cd projects
```

Here, `projects` is a relative path because it depends on your current folder.

Simple meaning:

Relative path = location from where you are now.

10. Parent Directory Using `..`

The symbol `..` means:

one folder back

or

parent folder

Command:

```
cd ..
```

Example:

If you are inside:

```
/home/maleesha/my-project
```

and you run:

```
cd ..
```

You move back to:

```
/home/maleesha
```

Simple meaning:

`..` means go back/up one folder.

11. `cd` Command

The command `cd` means:

change directory

A directory means a folder.

So `cd` means:

change folder

Command:

```
cd folder_name
```

Example:

```
cd my-project
```

This moves the terminal into the `my-project` folder.

Simple meaning:

`cd` is used to move between folders.

12. Common `cd` Examples

Move into a folder

```
cd my-project
```

Meaning:

Move into the folder called `my-project`.

Move back one folder

```
cd ..
```

Meaning:

Move to the parent folder.

Move to the home folder

```
cd ~
```

Meaning:

Move to your home folder.

Important note

`cd` does not create a folder.

It only moves into an existing folder.

Wrong idea:

```
cd new-folder
```

This will not create `new-folder` if it does not already exist.

To create a folder, use:

```
mkdir new-folder
```

13. `ls` Command

The command `ls` means:

list

It shows the files and folders inside your current working directory.

Command:

```
ls
```

Example output:

```
index.html  
readme.md  
images
```

Simple meaning:

`ls` **shows what is inside the current folder.**

Use `ls` often to check your files before moving, copying, or deleting anything.

14. Hidden Files

Some files are hidden by default.

In Unix-like systems, hidden files usually start with a dot `.`.

Examples:

```
.git  
.env  
.config
```

These files may not appear when you run only:

```
ls
```

To show hidden files, use:

```
ls -a
```

Simple meaning:

Hidden files are files that usually start with `.` and are not shown by normal `ls`.

15. `ls -a`

The command:

```
ls -a
```

shows all files, including hidden files.

Here, `-a` means:

all

Simple meaning:

`ls -a` **shows normal files and hidden files.**

Example:

```
ls -a
```

Possible output:

```
.  
..  
.git  
.env
```

```
index.html
readme.md
```

What are `.` and `..`?

- `.` means current folder
- `..` means parent folder

16. `ls -l`

The command:

```
ls -l
```

shows a longer and more detailed list.

It may show information such as:

- File permissions
- Owner
- Group
- File size
- Date modified
- File or folder name

Simple meaning:

`ls -l` shows more details about files and folders.

17. Combining `ls` Flags

You can combine short flags.

Example:

```
ls -al
```

This means:

```
ls -a -l
```

So it shows:

- All files, including hidden files
- Long detailed information

Simple meaning:

```
ls -al
```

 shows all files with details.

18. `mkdir` Command

The command `mkdir` means:

make directory

A directory means a folder.

So `mkdir` means:

make folder

Command:

```
mkdir folder_name
```

Example:

```
mkdir sprint3-practice
```

This creates a new folder called `sprint3-practice`.

Simple meaning:

```
mkdir
```

 creates a folder.

19. Creating and Entering a Folder

You can create a folder and then move into it.

Step 1: Create folder

```
mkdir sprint3-practice
```

Step 2: Enter folder

```
cd sprint3-practice
```

Step 3: Check where you are

```
pwd
```

Simple meaning:

Use `mkdir` to create the folder, then use `cd` to enter it.

20. `touch` Command

The command `touch` can create an empty file.

Command:

```
touch filename
```

Example:

```
touch readme.md
```

This creates an empty file called `readme.md`.

Simple meaning:

`touch` creates a file if the file does not already exist.

Important note

If the file already exists, `touch` does not normally delete its content.

It may update the file's modified time.

For beginners, remember:

Use `touch` to create a new empty file.

21. What Is `echo` ?

The command `echo` prints text to the terminal.

Command:

```
echo "Hello"
```

Output:

```
Hello
```

Simple meaning:

`echo` **displays text.**

In Sprint 3, you also use `echo` to write text into files.

22. Output Redirection

Output means the result shown by a command.

Redirection means sending that result somewhere else.

In Sprint 3, you send text into a file using:

- `>`
- `>>`

Simple meaning:

Output redirection lets you send command output into a file.

23. Writing Text into a File Using `>`

The symbol `>` writes output into a file.

Example:

```
echo "Naomi was here." > readme.md
```

This writes the text into `readme.md`.

Important:

`>` will create the file if it does not exist.

But if the file already has content, `>` will replace the old content.

Simple meaning:

`>` means write into a file and overwrite old content.

Example

```
echo "First line" > notes.md
```

Now `notes.md` contains:

```
First line
```

If you run:

```
echo "Second line" > notes.md
```

Now the file contains only:

```
Second line
```

The first line is gone because `>` overwrote the file.

24. Appending Text into a File Using >>

The symbol >> adds output to the end of a file.

Example:

```
echo "More text" >> readme.md
```

This adds the text to the end of `readme.md`.

It does not remove the old content.

Simple meaning:

>> means add new text to the end of the file.

Example

```
echo "First line" > notes.md
```

Then:

```
echo "Second line" >> notes.md
```

Now `notes.md` contains:

```
First line
Second line
```

25. Difference Between > and >>

Symbol	Meaning	What happens to old content?
>	Write / overwrite	Old content is replaced
>>	Append	Old content stays

Simple meaning:

- > = replace the file content

- `>>` = add to the end of the file

Beginner warning:

Be careful with `>` because it can remove old content from a file.

26. `cat` Command

The command `cat` shows the content of a file in the terminal.

Command:

```
cat filename
```

Example:

```
cat readme.md
```

This prints the content of `readme.md` in the terminal.

Simple meaning:

`cat` **quickly shows what is inside a file.**

Use `cat` for small files.

27. `less` Command

The command `less` opens a file in a scrollable view.

Command:

```
less filename
```

Example:

```
less readme.md
```

This lets you read a longer file inside the terminal.

To exit `less`, press:

q

Simple meaning:

`less` is used to read longer files more comfortably.

28. Difference Between `cat` and `less`

Command	Best For	Simple Meaning
<code>cat file</code>	Small files	Prints the whole file quickly
<code>less file</code>	Longer files	Opens a scrollable reading view

Simple rule:

Use `cat` for short files.

Use `less` for long files.

29. `mv` Command

The command `mv` means:

move

It can do two things:

1. Rename a file or folder
2. Move a file or folder to another place

Simple meaning:

`mv` is used to rename or move files and folders.

30. Rename a File with `mv`

Command:

```
mv oldname newname
```

Example:

```
mv readme.md notes.md
```

This renames `readme.md` to `notes.md`.

Simple meaning:

```
mv oldname newname
```

renames a file.

31. Move a File into a Folder with `mv`

Example:

```
mkdir docs  
mv notes.md docs/
```

Meaning:

- `mkdir docs` creates a folder called `docs`
- `mv notes.md docs/` moves `notes.md` into the `docs` folder

Simple meaning:

```
mv file folder/
```

moves a file into a folder.

32. `cp` Command

The command `cp` means:

copy

It copies a file or folder.

Command:

```
cp source destination
```

Example:

```
cp notes.md notes-copy.md
```

This creates a copy of `notes.md` called `notes-copy.md`.

Simple meaning:

`cp` **copies a file.**

33. Copying a Folder with `cp -r`

To copy a folder, use:

```
cp -r source_folder destination_folder
```

Example:

```
cp -r docs docs-backup
```

This copies the `docs` folder and creates another folder called `docs-backup`.

Here, `-r` means recursive.

Simple meaning:

Use `cp -r` when copying folders.

What does recursive mean?

Recursive means the command goes inside the folder and copies everything inside it.

For beginners:

Folder copy needs `-r` because a folder may contain many files and subfolders.

34. `rm` Command

The command `rm` means:

remove

It deletes files.

Command:

```
rm filename
```

Example:

```
rm notes-copy.md
```

This deletes the file called `notes-copy.md`.

Simple meaning:

`rm` **deletes a file.**

35. Be Careful with `rm`

`rm` can be dangerous if used incorrectly.

In many Bash environments, files deleted with `rm` may not go to the Recycle Bin.

They may be removed directly.

Simple meaning:

Use `rm` carefully because deleted files may be hard to recover.

Good beginner habit:

Before deleting, run:

```
pwd
```

Then run:

```
ls
```

Then delete only the correct file:

```
rm filename
```

36. Delete a Folder with `rm -r`

To delete a folder and its contents, use:

```
rm -r folder_name
```

Example:

```
rm -r docs-backup
```

This deletes the `docs-backup` folder and the files inside it.

Here, `-r` means recursive.

Simple meaning:

`rm -r` **deletes a folder and everything inside it.**

Beginner warning:

Be very careful with `rm -r` because it can delete many files at once.

37. Command Arguments

An argument is extra information given to a command.

Example:

```
cd my-project
```

Here:

- `cd` is the command
- `my-project` is the argument

The argument tells `cd` which folder to enter.

Another example:

```
cat readme.md
```

Here:

- `cat` is the command
- `readme.md` is the argument

Simple meaning:

An argument tells the command what to work on.

38. Options and Flags

A command can have extra settings.

These settings are called:

- Options
- Flags

Example:

```
ls
```

This shows normal files.

But:

```
ls -a
```

This shows all files, including hidden files.

Here, `-a` is a flag.

Simple meaning:

Flags change how a command behaves.

39. Short Flags

Short flags usually use one hyphen.

Examples:

```
ls -a
```

```
ls -l
```

```
rm -r folder_name
```

Simple meaning:

Short flags use one hyphen, like `-a`, `-l`, and `-r`.

40. Combining Short Flags

Some short flags can be combined.

Example:

```
ls -al
```

This is the same as:

```
ls -a -l
```

Simple meaning:

`ls -al` combines `-a` and `-l`.

41. Long Options

Long options usually use two hyphens.

Examples:

```
ls --help
```

```
ls --version
```

Simple meaning:

Long options use two hyphens and are usually easier to read.

42. Long Options with Values

Some long options can receive values.

Example:

```
ls --width=50
```

This sets the output width to `50`.

Simple meaning:

Some long options use `=` to pass a value.

43. Short Options with Values

Some short options can also receive values.

Example:

```
ls -w 50
```

This also sets the output width to `50`.

Simple meaning:

Some short options use a space before the value.

44. Examples of `ls` Options from Sprint 3

```
ls --color=never
```

```
ls --color=never
```

This tells `ls` not to use color in the output.

Simple meaning:

Show the list without color.

```
ls -w 50
```

```
ls -w 50
```

This sets the output width to `50`.

Simple meaning:

Show the list using a width of 50.

```
ls --width=50
```

```
ls --width=50
```

This is the long option version of `ls -w 50`.

Simple meaning:

`--width=50` is the long option version of `-w 50`.

45. `--help` Option

Many commands support:

```
--help
```

Example:

```
ls --help
```

This shows help information about the `ls` command.

Simple meaning:

`--help` shows how to use a command.

Use `--help` when you forget how a command works.

46. `--version` Option

Many commands support:

```
--version
```

Example:

```
ls --version
```

This shows the version of the command or program.

Simple meaning:

`--version` shows which version of a command/program is installed.

47. `man` Command

The command `man` means:

manual

It opens the manual page for a command.

Command:

```
man command_name
```

Example:

```
man ls
```

Another example:

```
man mkdir
```

Simple meaning:

`man` opens the instruction manual for a command.

To exit the manual page, press:

```
q
```

48. Difference Between `--help` and `man`

Command	Simple Meaning
<code>command --help</code>	Shows quick help
<code>man command</code>	Opens the full manual page

Simple rule:

Use `--help` for quick help.

Use `man` for more detailed help.

49. Sprint 3 Command Summary

Command	Simple Meaning	Example
<code>pwd</code>	Show current folder	<code>pwd</code>
<code>cd</code>	Change folder	<code>cd my-project</code>
<code>cd ..</code>	Go back one folder	<code>cd ..</code>
<code>cd ~</code>	Go to home folder	<code>cd ~</code>
<code>ls</code>	List files/folders	<code>ls</code>
<code>ls -a</code>	Show hidden files too	<code>ls -a</code>
<code>ls -l</code>	Show detailed list	<code>ls -l</code>
<code>mkdir</code>	Create folder	<code>mkdir sprint3-practice</code>
<code>touch</code>	Create file	<code>touch readme.md</code>
<code>echo</code>	Print text	<code>echo "Hello"</code>
<code>></code>	Write and overwrite	<code>echo "Hi" > readme.md</code>
<code>>></code>	Append to file	<code>echo "More" >> readme.md</code>
<code>cat</code>	Show file content	<code>cat readme.md</code>
<code>less</code>	Read longer file	<code>less readme.md</code>
<code>mv</code>	Move or rename	<code>mv old.md new.md</code>
<code>cp</code>	Copy file	<code>cp notes.md copy.md</code>
<code>cp -r</code>	Copy folder	<code>cp -r docs docs-backup</code>
<code>rm</code>	Delete file	<code>rm copy.md</code>
<code>rm -r</code>	Delete folder	<code>rm -r docs-backup</code>
<code>man</code>	Open manual	<code>man ls</code>
<code>--help</code>	Show command help	<code>ls --help</code>
<code>--version</code>	Show version	<code>ls --version</code>

50. Sprint 3 Practice Task: Full Walkthrough

Complete this task step by step.

Step 1: Check where you are

```
pwd
```

Meaning:

Show your current folder.

Step 2: Create a practice folder

```
mkdir sprint3-practice
```

Meaning:

Create a folder called `sprint3-practice`.

Step 3: Move into the practice folder

```
cd sprint3-practice
```

Meaning:

Enter the `sprint3-practice` folder.

Step 4: Create a file

```
touch readme.md
```

Meaning:

Create an empty file called `readme.md`.

Step 5: Write text into the file

```
echo "I am learning Bash file commands." > readme.md
```

Meaning:

Write this text into `readme.md`.

This uses `>`, so it writes and overwrites.

Step 6: Add more text to the file

```
echo "This is Sprint 3 practice." >> readme.md
```

Meaning:

Add another line to the end of `readme.md`.

This uses `>>`, so it appends.

Step 7: Show the file content

```
cat readme.md
```

Expected output:

```
I am learning Bash file commands.  
This is Sprint 3 practice.
```

Meaning:

Show what is inside the file.

Step 8: Rename the file

```
mv readme.md notes.md
```

Meaning:

Rename `readme.md` to `notes.md`.

Step 9: Copy the file

```
cp notes.md notes-copy.md
```

Meaning:

Create a copy of `notes.md` called `notes-copy.md`.

Step 10: List files with details

```
ls -l
```

Meaning:

Show files with detailed information.

You should see files like:

```
notes.md
notes-copy.md
```

Step 11: Delete the copied file

```
rm notes-copy.md
```

Meaning:

Delete only the copied file.

Step 12: Check remaining files

```
ls
```

Expected result:

```
notes.md
```

Meaning:

Only the original renamed file should remain.

51. Sprint 3 Checkpoint

Your checkpoint is:

Create a folder, create a file inside it, add text, append text, show the file, rename it, copy it, and delete the copy.

You should be able to do this using commands.

Required command flow:

```
pwd
mkdir sprint3-practice
cd sprint3-practice
touch readme.md
echo "I am learning Bash file commands." > readme.md
echo "This is Sprint 3 practice." >> readme.md
cat readme.md
mv readme.md notes.md
cp notes.md notes-copy.md
ls -l
rm notes-copy.md
ls
```

Simple meaning:

This checkpoint proves that you can manage basic files and folders from the terminal.

52. Common Beginner Mistakes

Mistake 1: Not checking where you are

Problem:

You create or delete files in the wrong folder.

Better habit:

```
pwd
```

Simple meaning:

Always check your location before important commands.

Mistake 2: Using `>` when you meant `>>`

Problem:

You accidentally overwrite existing content.

Remember:

```
>
```

means overwrite.

```
>>
```

means append.

Simple meaning:

Use `>>` when you want to keep old text and add new text.

Mistake 3: Deleting the wrong file

Problem:

You use `rm` on the wrong file.

Better habit:

```
pwd
ls
rm correct-file-name
```

Simple meaning:

Check first, delete second.

Mistake 4: Forgetting spaces in commands

Wrong:

```
cdmy-project
```

Correct:

```
cd my-project
```

Simple meaning:

Commands need spaces between the command and the argument.

Mistake 5: Confusing files and folders

File copy:

```
cp file1 file2
```

Folder copy:

```
cp -r folder1 folder2
```

Simple meaning:

Files and folders sometimes need different command options.

Mistake 6: Thinking `cd` creates folders

Wrong idea:

```
cd new-folder
```

This does not create a folder.

Correct:

```
mkdir new-folder  
cd new-folder
```

Simple meaning:

Use `mkdir` to create. Use `cd` to enter.

Mistake 7: Forgetting that `rm` may not use Recycle Bin

Problem:

You delete a file and expect it to be in the Recycle Bin.

In many Bash environments, `rm` removes directly.

Simple meaning:

Be careful because `rm` can permanently remove files.

53. Key Terms Summary

Term	Simple Meaning
Bash	A common shell used to run commands
Working directory	The folder where the terminal is currently standing
Path	Address of a file or folder
Absolute path	Full address from the root of the system
Relative path	Address based on where you are now
Parent directory	The folder one level above the current folder
<code>..</code>	Means parent folder / go one folder up
Directory	Another word for folder
File	A saved item, such as <code>readme.md</code>
Command	An instruction typed into the terminal

Term	Simple Meaning
Argument	Information given to a command
Option	Extra setting for a command
Flag	Short option that changes command behavior
Hidden file	A file usually starting with <code>.</code>
Output	Result produced by a command
Redirection	Sending command output somewhere else
Overwrite	Replace old content with new content
Append	Add new content to the end
Recursive	Work through a folder and its contents
Manual page	Help page for a command

54. Final Sprint 3 Understanding

After Sprint 3, you should understand that the terminal is not only for typing simple text commands.

You can use the terminal to manage real project files and folders.

You should now understand how to:

- Find where you are using `pwd`
- Move between folders using `cd`
- Go back using `cd ..`
- List files using `ls`
- Show hidden files using `ls -a`
- Show detailed file information using `ls -l`
- Create folders using `mkdir`
- Create files using `touch`
- Write text using `>`
- Append text using `>>`
- Read files using `cat` and `less`
- Rename files using `mv`
- Move files using `mv`
- Copy files using `cp`
- Copy folders using `cp -r`
- Delete files using `rm`
- Delete folders using `rm -r`
- Use flags and options

- Use `--help` and `man` when you need help

Simple final meaning:

Sprint 3 gives you the file and folder control skills needed before Bash scripting, Git, and database projects.

55. Sprint 3 Mini Assessment

Answer these questions before moving to Sprint 4.

1. What does `pwd` do?
2. What is a working directory?
3. What does `cd` do?
4. What does `cd ..` mean?
5. What does `cd ~` do?
6. What is a path?
7. What is an absolute path?
8. What is a relative path?
9. What does `ls` do?
10. What does `ls -a` show?
11. What does `ls -l` show?
12. What does `mkdir` do?
13. What does `touch` do?
14. What does `echo` do?
15. What is output redirection?
16. What is the difference between `>` and `>>`?
17. What does `cat` do?
18. What does `less` do?
19. How do you exit `less`?
20. What does `mv` do?
21. How do you rename a file?
22. How do you move a file into a folder?
23. What does `cp` do?
24. How do you copy a folder?
25. What does `rm` do?
26. Why should you be careful with `rm`?
27. What does `rm -r` do?
28. What is an argument?
29. What is a flag?
30. What is the difference between short flags and long options?
31. What does `--help` do?
32. What does `--version` do?
33. What does `man` do?
34. How do you exit a manual page?

56. Sprint 3 Completion Checklist

Tick these when completed:

- ☐ I can check my current folder using `pwd`.
- ☐ I can explain what working directory means.
- ☐ I can move into a folder using `cd`.
- ☐ I can go back one folder using `cd ..`.
- ☐ I can go to my home folder using `cd ~`.
- ☐ I can explain absolute path and relative path.
- ☐ I can list files using `ls`.
- ☐ I can show hidden files using `ls -a`.
- ☐ I can show detailed file information using `ls -l`.
- ☐ I can create a folder using `mkdir`.
- ☐ I can create a file using `touch`.
- ☐ I can print text using `echo`.
- ☐ I can write text into a file using `>`.
- ☐ I understand that `>` overwrites old content.
- ☐ I can append text using `>>`.
- ☐ I understand that `>>` keeps old content.
- ☐ I can read a small file using `cat`.
- ☐ I can read a longer file using `less`.
- ☐ I can exit `less` using `q`.
- ☐ I can rename a file using `mv`.
- ☐ I can move a file into a folder using `mv`.
- ☐ I can copy a file using `cp`.
- ☐ I can copy a folder using `cp -r`.
- ☐ I can delete a test file using `rm`.
- ☐ I understand that `rm` must be used carefully.
- ☐ I can delete a test folder using `rm -r`.
- ☐ I understand that `rm -r` must be used very carefully.
- ☐ I understand what command arguments are.
- ☐ I understand basic flags like `-a`, `-l`, and `-r`.
- ☐ I can use `--help` to get quick help.
- ☐ I can use `man` to open a manual page.
- ☐ I completed the Sprint 3 checkpoint task.

57. End of Sprint 3 Notes

You are ready to move to **Sprint 4: Bash Scripting Basics** only after you can complete the Sprint 3 checkpoint confidently.

Before moving on, make sure you can do this without copying every command blindly:

```
pwd
mkdir sprint3-practice
cd sprint3-practice
touch readme.md
echo "I am learning Bash file commands." > readme.md
echo "This is Sprint 3 practice." >> readme.md
cat readme.md
mv readme.md notes.md
cp notes.md notes-copy.md
ls -l
rm notes-copy.md
ls
```

Final simple meaning:

Sprint 3 teaches you how to control files and folders using Bash. This is the practical foundation for Bash scripting, Git, PostgreSQL, and SQL project work.